

Lab Report: Containers and Kubernetes

Name: Robert Mesek

Lab: 6

Date: April 24, 2026

Task 1: The "Greeter" Container

1. Create a `greeter.sh` script that prints "hello world!" to stdout and ensure it has the executable bit set.
2. Create a Dockerfile based on a different Linux distribution than your host machine/VM.
3. Include the `greeter.sh` script in the image and build it.
4. Test the container in interactive shell mode and verify the distribution (e.g., using `lsb_release -a`).
5. Run the container by calling the script as an executable and capture the output.

Solution

`greeter.sh`

```
#!/bin/sh
echo "hello world!"
```

Dockerfile

```
FROM alpine:latest

WORKDIR /app

COPY greeter.sh .

CMD ["/greeter.sh"]
```

```
% docker build -t my-greeter .
...
```

```
% docker run -it my-greeter /bin/sh
# cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.23.4
```

```
PRETTY_NAME="Alpine Linux v3.23"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
```

```
% docker run my-greeter /app/greeter.sh
hello world!
```

Task 2: Web Server Container

1. Create a Dockerfile based on an image that includes a web server (e.g., [httpd](#)).
2. Include a custom HTML page in the container.
3. Build and start the container, providing verification that the HTTP port is accessible and responding with your custom content.

Solution

Dockerfile

```
FROM httpd:latest

COPY ./index.html /usr/local/apache2/htdocs/
```

index.html

```
<html>
<body>
  <h1>Hello world!</h1>
</body>
</html>
```

```
% docker build -t my-web-server .
...
% docker run -d --name lab-web-server -p 8080:80 my-web-server
```

```
% curl http://localhost:8080
<html>
<body>
  <h1>Hello world!</h1>
</body>
</html>
```

```
% docker rm -f lab-web-server
lab-web-server
```

Task 3: K8s Cluster Deployment

1. Start a local development cluster using `minikube`.
2. Configure your environment to use the internal minikube Docker daemon using `$(minikube docker-env)` and rebuild your previous images so they are available to the cluster.
3. Create a YAML deployment file for the "web server" container.
4. Verify that the web server is running within the Kubernetes cluster and responding to requests, ensuring the access port is correctly declared.
5. Consider the use of `imagePullPolicy: Never` and why it might be necessary when working with local images.

Solution

`web-server-deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-server-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-server
  template:
    metadata:
      labels:
        app: web-server
    spec:
      containers:
        - name: apache-container
          image: my-web-server:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-server-service
spec:
  type: LoadBalancer
  selector:
    app: web-server
  ports:
    - protocol: TCP
```

```
port: 8888
targetPort: 80
```

```
% kubectl apply -f web-server-deployment.yaml
deployment.apps/web-server-deployment created
service/web-server-service created
```

```
% kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web-server-deployment-598d6f484f-m4cts	1/1	Running	0	19s
web-server-deployment-598d6f484f-q772w	1/1	Running	0	19s

```
% curl http://localhost:8888
<html>
<body>
  <h1>Hello world!</h1>
</body>
</html>
```

```
% kubectl delete -f web-server-deployment.yaml
deployment.apps "web-server-deployment" deleted
service "web-server-service" deleted
```

References

- [Docker Documentation](#)
- [Kubernetes Documentation](#)
- [Minikube Documentation](#)